

פרויקט גמר מערכות משובצות מחשב

כספת חכמה



מגישה

שירה בקשי

בהנחיית: בת ציון שולביץ

שנה"ל תשפ"ד

תוכן עניינים

1	הצהרת סטודנט/ים	2
2	מבוא	3
3	תקציר הפרויקט	4
3.1	תרשים מלבנים	5
3.2	הסבר פעולת המערכת	6
4	שרטוט חשמלי	7
5	הבקר ARDUINO UNO	8
6	LCD צבעוני גרפי	9
6.1	קביעת צבע לכל פיקסל	10
6.2	מסך TOUCH	13
7	מנוע סרוו	15
7.1	מתח הפעלה של מנועי סרבו	15
7.2	חיבורי מנוע הסרבו	16
7.3	קטע תוכנית לדוגמא:	16
8	קורא טביעת אצבע	17
8.1	חיבור הקורא למעבד	19
8.2	תוכנית לקבלת טביעת האצבע	19
9	תיאור פרויקט	23
9.1	הסבר על הפרויקט	23
9.2	צילומי מסך + הסבר	23
10	תיעוד הפרויקט	24
10.1	תהליך עבודה	24
10.2	תקלות ופתרונם	24
11	תוכנה 25	
11.1	תרשים זרימה	25
1.2	תוכניות	26
11.3	הסבר פונקציות	37
12	סיכום	39
13	תקשורת SPI	40

1 הצהרת סטודנט/ים

סטודנט: שירה בקשי ת.ז: 325703486

אני/אנו הח"מ, מצהיר בזאת כי פרויקט הגמר וספר הפרויקט המצ"ב נעשו על ידי/דינו בלבד.

פרויקט הגמר נעשה על סמך הנושאים שלמדתי בבית הספר ובאופן עצמאי.

פרויקט הגמר וספר הפרויקט נעשו על בסיס הנחייתו של המנחה האישי.

מקורות המידע בהם השתמשתי לביצוע פרויקט הגמר מצוינים ברשימת המקורות המצוינים בספר הפרויקט.

אני מודע לאחריות שהנני מקבל על עצמי על ידי חתימתי על הצהרה זו שכל הנאמר בה אמת ורק אמת.

אישור המנחה האישי

הריני מאשר שהפרויקט בוצע בהנחייתי, קראתי את ספר הפרויקט ומצאתי כי הוא מוכן לצורך הגשת הסטודנט להגנה על פרויקט גמר.

שם המנחה: _____ בת ציון שולביץ חתימה: _____ תאריך: _____

2 מבוא

במסגרת שיעורי מגמת הנדסת אלקטרוניקה ומחשבים, התבקשתי להכין פרויקט שמסכם את הנושאים שנלמדו במהלך שנות הלימוד במגמה.

תיאור הפרויקט:

הפרויקט מבוסס על טכנולוגיות ביומטריות ואלקטרוניות לשמירה על ביטחון חפצים באמצעות זיהוי טביעת אצבע ובקרת גישה דיגיטלית. תהליך הפיתוח כולל עמידות בפני ניסיונות פריצה ויכולת שימוש נוחה למשתמשים. פרויקט זה יכול לשפר את ביטחון האישי ולהציע פתרונות מתקדמים בהגנה חפצים יקרים.

למדתי כיצד לעבוד עם הרכיבים הנדרשים לפרויקט, איך להפעיל אותם ועל האופן שבו הם פועלים.

המערכות והרכיבים ששילבתי בפרויקט:

• בקר Arduino Uno

• finger print

• מנוע servo

• led

למדתי להשתמש בתוכנת ה-Arduino באמצעותה כתבתי את התוכנית לפרויקט.

3 תקציר הפרויקט

הפרויקט מבצע פתיחה וסגירה של כספת בעזרת טביעת אצבע וסיסמה.

הפרויקט כולל חלקים ובו אופציה לשתי פעולות:

1. קליטת משתמש בפעם הראשונה – לקיחת טביעת האצבע שלו, וסיסמה.

2. הכנסת סיסמה או טביעת אצבע לפתיחת הכספת.

הרחבה:

1. המשתמש מזין סיסמה והמערכת מאמתת אותה ופותחת את הכספת.

2. המשתמש מכניס טביעת אצבע המערכת מאמתת ופותחת את הכספת.

כאשר האימות מצליח נורת לד נדלקת בכחול, ואם נכשל נדלקת באדום.

הסיבה לבחירת הפרויקט:

בעקבות גנבות רבות באזור מגוריי, עלה המחשבה שכדי להגן על חפצים יקרי ערך, יש לשמור עליהם בצורה מאובטחת יותר.

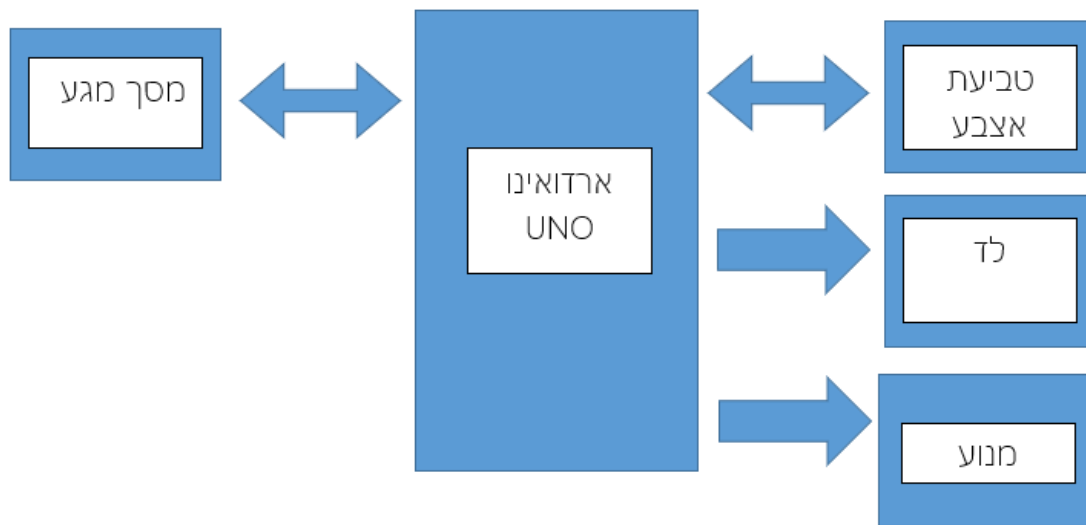
לפרויקט הנ"ל ניתן לפתח לכספת אמצעי זיהוי רבים יותר ומאובטחים יותר. לדוגמא זיהוי גלגל העין, בנוסף אפשר לשפר את מפתח הכספת שיהיה חזק ומסיבי ובלתי ניתן לפריצה.

בתחילה חשבתי לעשות כך את הפרויקט:

פתיחת הכספת תהיה על ידי טביעת אצבע וקוד. במידה ופורצים לכספת, בעל הכספת מקבל התראה בנייד, (שיחה או הודעה) והכספת מדליקה רטט חזק כדי לעכב אותו מלפרוץ.

אך לאחר ביצוע ראיתי לנכון שבמידה ומנסים לפרוץ (סיסמאות שגויות, טביעת אצבע לא נכונה) הכספת לא תפתח ותידלק נורה אדומה.

3.1 תרשים מלבנים



3.2 הסבר פעולת המערכת

הבקר - התוכנה למעשה צרובה בבקר ומחברת בין כל חלקי הפרויקט. חיבור הרכיבים לבקר מתבצע באמצעות מספר פרוטוקולים.

תצוגה גרפית מסך מגע – תצוגה גרפית צבעונית 320 x 240 פיקסלים, על התצוגה אנחנו מקבלים את כל הנתונים שהמערכת מייצרת

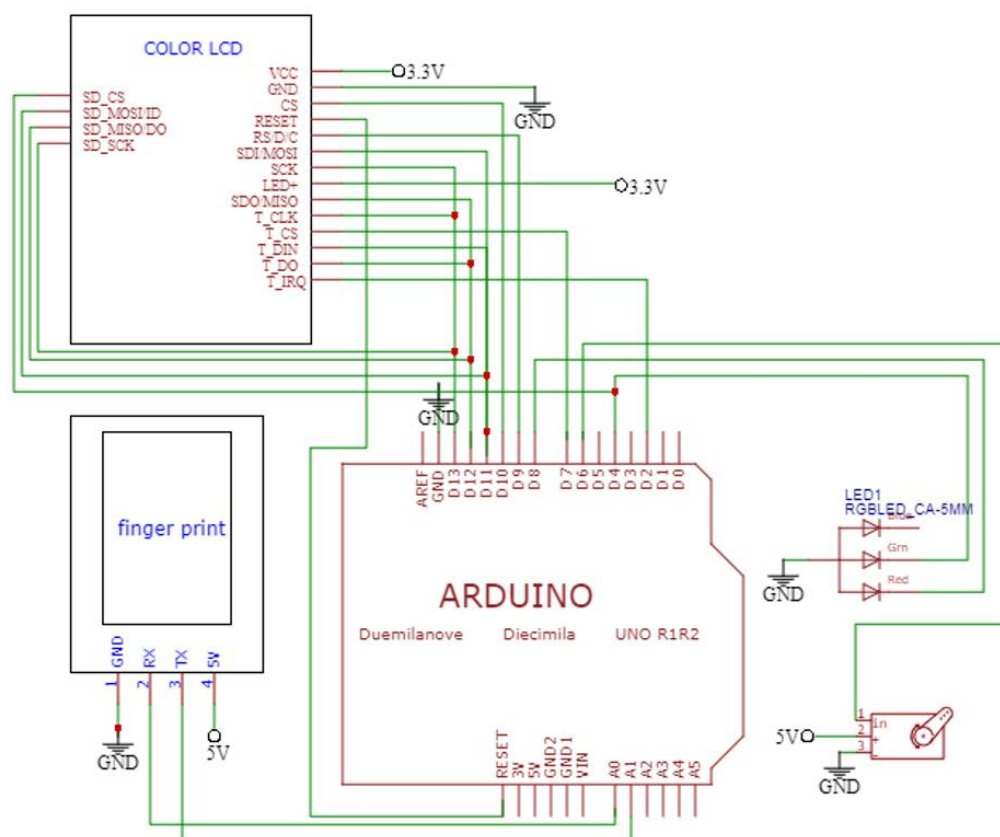
חיישן טביעת אצבע - סורק את טביעת האצבע שמונחת עליו לפי קטע

תכנית. .

• סרוו - מנוע המופעל באופן סיבובי או לינארי המאפשר שליטה מדויקת, הוא

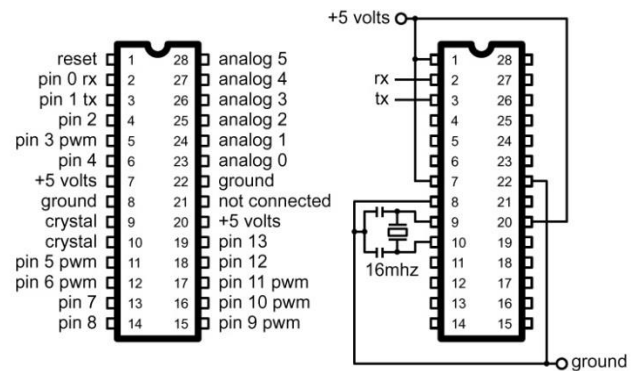
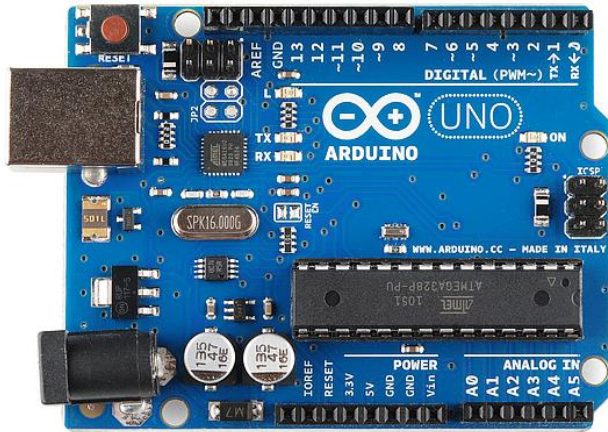
מורכב ממנוע מתאים המחובר.

4 שרטוט חשמלי



5 הבקר ARDUINO UNO

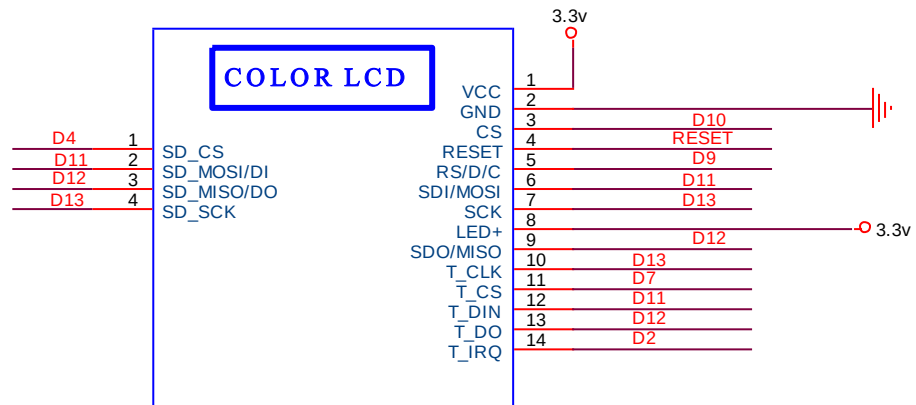
מבוסס על המיקרו בקר Atmega328p



המאפיינים העיקריים של הרכיב Atmega328p הם:

- תדר שעון 16MHZ.
- מתח עבודה 5v (אספקת מתח לכרטיס 7v-12v).
- זרם בהדקי I/O עד 40mA.
- זיכרון תוכנית (flash) בגודל 32k.
- זיכרון נתונים (ram) בגודל 2k.
- 14 כניסות ויציאות דיגיטליות.
- 6 יציאות PWM.
- 6 כניסות אנאלוגיות ברזולוציה של 10 סיביות.
- תקשורת טורית (rs232, i2c, spi).
- 2 פסיקות חיזוניות.

6 LCD צבעוני גרפי



LCD גרפי צבעוני, ברזולוציה של 240x320 פיקסלים RGB

כל פיקסל מקבל ערך של 16bit או 18bit המרכיבים את הצבע.

שליחת המידע היא באמצעות תקשורת טורית SPI המורכב מ-4 הדקים: MOSI, MISO, CS, SCK (ניתן לחבר 3 מערכות: צג, מסך מגע, כרטיס זיכרון).

ועוד שני רגלי בקרה: הדק RS לקביעת נתון או בקרה ו-RESET לאיפוס.

תפקיד הפינים:

Reset - מאתחל את מעבד התצוגה.

AD/RS - הדק הקובע אם נשלח מילת בקרה (כתובת פיקסל) RS=0

או קובע אם נשלח מידע של הצבע RS=1.

MOSI/SDI - קו כניסת נתונים טורי.

MISO/SDO - קו יציאת נתונים טורי.

SCK - קו שעון ששולח המעבד החיצוני.

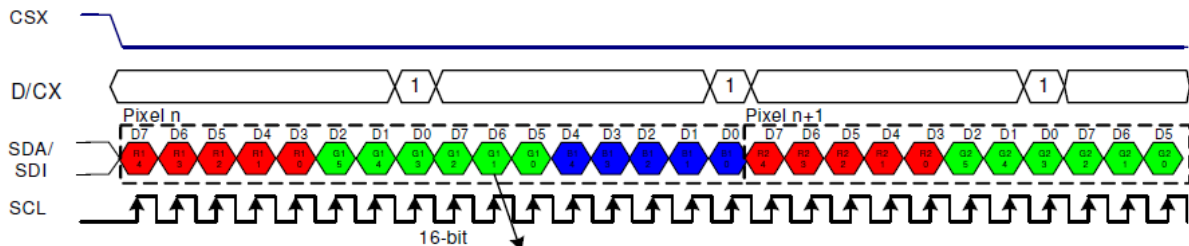
CS - הדק אפשר תקשורת. כאשר מתחילה תקשורת, ההדק יורד ל-0.

6.1 קביעת צבע לכל פיקסל

ניתן לשלוח מידע על צבע הפיקסל לפי 16bit ואז שולחים מידע של שני בתים או לפי 18bit ואז שולחים שלושה בתים וכל בית מכיל מידע על צבע ירוק, אדום או כחול.

מוד של 16bit

בתוך המילה המשודרת של 16Bit יש את כל הצבעים לפי התיאור הבא:



5 ביטים - מרכיב של צבע אדום.

6 ביטים - מרכיב של צבע ירוק.

5 ביטים - מרכיב של צבע כחול.

בעזרת שילוב של כל הצבעים ניתן לקבל 65K צבעים, כדי להגיע לכל פיקסל יש צורך לכתוב קודם את הכתובת ואחר כך את המידע.

לפי התרשים רואים שבהתחלת התקשורת $CS=0$ ולאחר מכן משדורים 16 ביטים בצורה טורית.

כל עליית שעון מועבר ביט אחד לפי הפרוטוקול.

כדי לקבוע אם שידור הנתונים יהיה כתובת או מידע נשנה את המצב הלוגי של RS.

תפקידה	פרמטרים שהפונקציה מקבלת או מחזירה	
Lcd.begin()	פונקציה המאפשרת להשתמש במסך ה-lcd.	אינה מקבלת או מחזירה פרמטרים.
LcdTouch.clearButton()	פונקציה המאפשרת להשתמש באופציית נגיעה במסך ה-lcd.	אינה מקבלת או מחזירה פרמטרים.
LcdTouch.setRotation(0)	פונקציה הקובעת את	מקבלת ערכים 0-3 ולא מחזירה.

	כיוון המסך.	)
מקבלת ערך של גודל פונט ולא מחזירה.	שינוי גודל הפונט.	LcdTouch.setTextSize (2)
מקבלת ערכי x,y ולא מחזירה.	פונקציה המקבלת שני ערכים x,y לבחירת מיקום על המסך.	LcdTouch.setCursor (40, 40)
מקבלת את הפרמטרים: xMin - ערך מינימלי על ציר ה-X. yMin - ערך מינימלי על ציר ה-Y. xMax - ערך מקסימלי על ציר ה-X. yMax - ערך מקסימלי על ציר ה-Y. אינה מחזירה ערך.	קובעת את תחום המגע של המסך, כלומר את גבולות הצירים האופקיים והאנכיים לזיהוי מגע.	LcdTouch.set(3780, 372, 489, 3811)
הפונקציה אינה מקבלת ומחזירה ערך בוליאני.	מזהה נגיעה על המסך.	LcdTouch.touched()
לא מקבלת פרמטרים. מחזירה ערך בוליאני אם המסך נלחץ או לא.	קריאת מיקום לחיצה.	LcdTouch.readTouch();
אינה מקבלת פרמטרים.	קריאת מיקום לחיצה על רוחב המסך.	LcdTouch.xTouch
אינה מקבלת פרמטרים.	קריאת מיקום על אורך המסך.	LcdTouch.yTouch
מקבלת את מיקום הלחיצה x,y ומחזירה את מספר הפקד.	בודקת את מספר הפקד שנלחץ.	LcdTouch.ButtonTouch(x, y)
מקבלת צבע מסך לפי 16 סיביות,	צביעת כל המסך.	LcdTouch.fillScreen

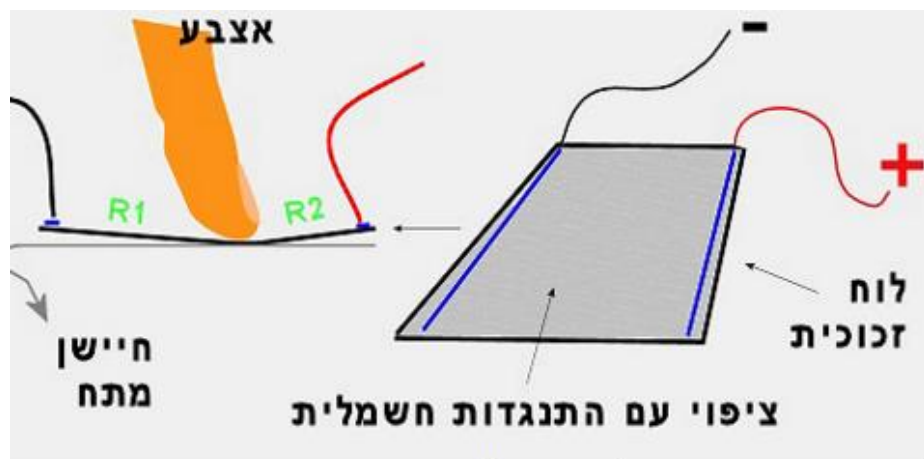
ולא מחזירה.		(WHITE)
מקבלת מיקום x,y על המסך , מחרוזת להדפסה, גודל פונט, וצבע הטקסט	הדפסת טקסט על המסך עם קביעת מיקום גודל הפונט, צבע שורה.	LcdTouch.print (10 ,20, "text", 2, BLACK)
מקבלת מחרוזת להדפסה ולא מחזירה.	מדפיסה הודעה כל המסך ויורדת שורה.	LcdTouch.println ("1");
מקבלת פרמטרים: d - מזהה ייחודי לכפתור. x - מיקום אופקי (ציר X) של הפינה השמאלית העליונה של הכפתור. y - מיקום אנכי (ציר Y) של הפינה השמאלית העליונה של הכפתור. w - רוחב הכפתור. h - גובה הכפתור. r - רדיוס הפינות המעוגלות של הכפתור. outlineColor - צבע המסגרת של הכפתור. fillColor - צבע המילוי של הכפתור. label - טקסט שמופיע על הכפתור. textSize - גודל הטקסט על הכפתור.	מציירת כפתור על המסך במיקום מסוים, עם צבע מסגרת, צבע מילוי וטקסט בתוכו.	LcdTouch.drawButton(1 , 120, 100, 90, 50, 10, BLACK, WHITE, "text", 2)
מקבלת שם צבע שהוגדר מראש .	קובעת את הצבע של	LcdTouch.setTextColor(

BLACK);	הטקסט.	וצובעת את הטקסט בצבע.
---------	--------	-----------------------

6.2 מסך TOUCH



עיקרון פעולה



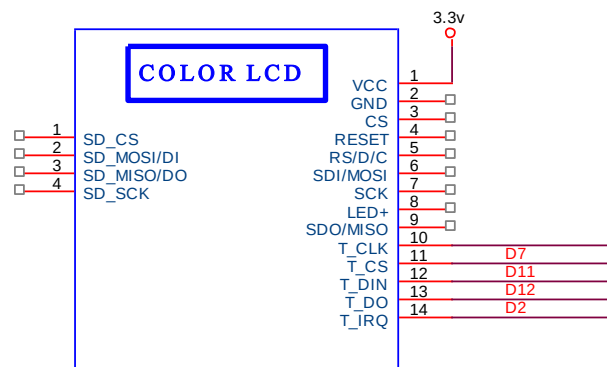
משטח מצופה התנגדות מכוסה בזכוכית דקה (כפי שמתואר באיור), בזמן נגיע במסך למעשה מתבצעת חלוקה של המסך לשני נגדים $R1, R2$ בציר Y . על מנת ליצור חלוקה גם בציר X , הכניסו משטח נוסף כך שמתבצעת חלוקה לשני נגדים גם בציר זה.

חלוקה לשני נגדים בכל נקודת מגע מאפשרת חלוקת מתח, המתח תמיד יהיה יחסי לנגיעה.

באמצעות ממיר ADC ברזולוציה של 12 סיביות דו ערוצי (Y, X) הנמצא על המסך אני ממירים את המתח למידע דיגיטלי הנשלח החוצה למעבד בתקשורת טורית SPI.

כעת נותר רק להתאים את התוצאות המתקבלות לכמות הפיקסלים על המסך.

חיבור חשמלי לבקר



הרגלים המסומנות ב-T הן האחראיות לתקשורת הטורית, 4 מהן בפרוטוקול SPI, T_IRQ יורד ל-'0' בזמן נגיעה על המסך. למעשה באמצעות תקשורת זאת אנו קוראים את ערכי הממיר לצירים X,Y מהמסך.

פונקציות לקריאת המיקום

ציר X $x = \text{lcdtouch.readx}();$

ציר Y $y = \text{lcdtouch.ready}();$

הפונקציות מחזירות את מיקום הלחיצה.

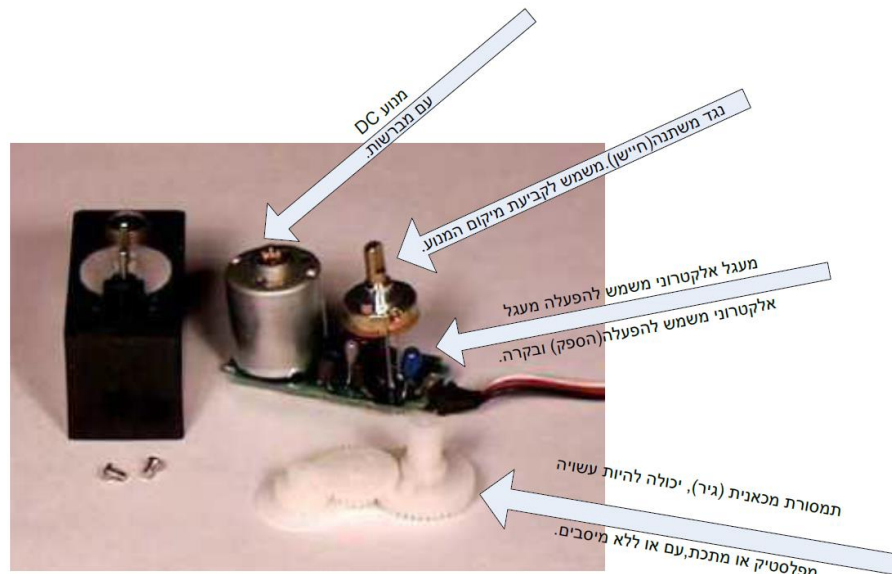
דוגמא לקטע תוכנית הממתין ללחיצת מקש וקריאת ערכי X,Y ובדיקה האם הלחיצה בוצעה באזור מוגדר.

```
if (LcdTouch.touched())
{
    LcdTouch.readTouch();
    x = LcdTouch.xTouch;
    y = LcdTouch.yTouch;

    int ButtonNum = LcdTouch.ButtonTouch(x, y);

    if (ButtonNum == 1)
    {
        screen1();
    }
    else if (ButtonNum == 2)
    {
        screen2();
    }
}
```

7 מנוע סרוו



הסבר - מעגל בקרה בחוג סגור. בכניסה נקבל פולס ברוחב T הזמן היחסי למתח, מושווה עם המתח שמגיע מהפוטנציומטר לאחר שעבר המרה במתמר זווית למתח. לאחר מכן מקבלים הפרש בין המתחים הללו שבעצם מהווה את השגיאה. המערכת מתקנת את השגיאה ובעצם מגיע למצב שהפרש המתחים הוא 0. ובכך מתבצעת פעולת בקרה בחוג סגור.

מנועי סרוו פועלים בחוג סגור המאפשר בקרה על מיקום המנוע (ופיצוי במקרה של שגיאה בין המיקום הרצוי ומיקום המנוע בפועל). הזמן שקובע את הזווית בתחומים בין $700 \sim 2300 \mu\text{sec}$ ובאמצעות הבקר ותוכנה מתאימה נקבע את הזמנים, בצורה מחזורית.

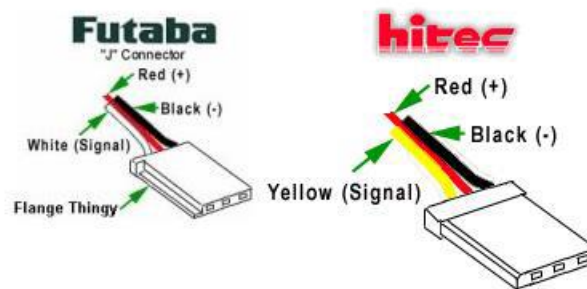
7.1 מתח הפעלה של מנועי סרבו

מנועי סרבו עובדים בטווח מתחים בדרך כלל בין 4.8 ל - 6 וולט. הסיבה לשימוש בסטנדרט זה היא הקרבה לרמת TTL (שהיא 5 וולט) שבה פועלים רוב המיקרו-מעבדים שמשמים לשליטה על מנועי סרבו.

7.2 חיבורי מנוע הסרבו

לכל מנועי הסרבו יש שלושה חוטים.

חוט שחור או חום שהוא אדמה (-), משותפת גם למתח ההספקה וגם לאות השליטה חוט אדום שהוא המתח (+), מספק את האנרגיה החשמלית להפעלת המנוע. חוט צהוב, כתום או לבן שהוא חוט האות לשליטה במנוע (אות הייחוס).



7.3 קטע תוכנית לדוגמא:

```
void Kavkav()  
{  
  myservo1.write(90);  
  delay(1000);  
  myservo1.write(180);  
  delay(1000);  
}
```

בפרוייקט זה נעשה שימוש בסרוו להדגמת פתיחת וסגירת הכספת. הסרבו יפתח ויסגר ל 90° .

8 קורא טביעת אצבע

הקדמה:

קורא טביעת אצבע מאפשר זיהוי טביעת אצבע ואחסון התבנית בבסיס הנתונים במחשב בכדי לאפשר זיהוי ואימות טביעת האצבע.

קיים מגוון רחב של קוראי טביעת אצבע הנבדלים בעיקר בסוג הסנסור (אופטי, קיבולי) הביומטרי ובאיכות הסנסור (רזולוציה ורגישות) בנוסף קוראי טביעת אצבע נבדלים באלגוריתם הפענוח של היצרן.

ככל שאלגוריתם הפענוח עצמו איכותי יותר, כך תתבצע השוואה טובה יותר בין טביעת האצבע שהניח העובד לבין טביעת האצבע הנמצאת בבסיס הנתונים. כל טביעת אצבע היא ייחודית טביעת אצבע היא הצורה המתקבלת מלחיצת אצבע על גבי משטח חלק.

הדגם הייחודי של טביעות האצבע נקבע בחודש הרביעי להתפתחות העובר. לכל אדם יש טביעות אצבע ייחודיות משלו אפילו לתאומים זהים יש טביעות אצבע שונות. ברוב המקרים הטביעות אינן נראות לעין אך ניתן לחשוף אותן באמצעות שימוש באמצעים כימיים שונים.

טביעת אצבע הייתה המאפיין הראשון ששימש לזיהוי ביומטרי והוא נמצא בשימוש בעיקר לצורכי כוחות הביטחון.

ברוב רשויות המשטרה בעולם נמצא מאגר טביעות אצבע של אנשים שנעצרו בגין עבירות כדי לזרז את זיהויים במקרה של פשע עתידי המאגר הממוחשב של

Integrated Automated Fingerprint Identification System

הוא מאגר טביעות האצבע הגדול בעולם ומכיל למעלה מ- 47 מיליון דגימות של טביעות אצבע ומשמש בנוסף גם לזיהוי קורבנות אסון.

ישנו הבדל מהותי בין שיטות הזיהוי הצבאיות והמשטרתיות לבין השיטות המסחריות. הפענוח במערכות הצבאיות והמשטרתיות מבוסס על ניתוח של נקודות ומאפיינים בטביעה, וביצוע השוואה של קידוד טביעה מול טביעה אחרת.

הפענוח במערכות האזרחיות מבוסס על השוואת צילומי טביעות אצבע אחת לשנייה. בשיטה זו רבות הטעויות ומידת הזיהוי אינה מוחלטת.

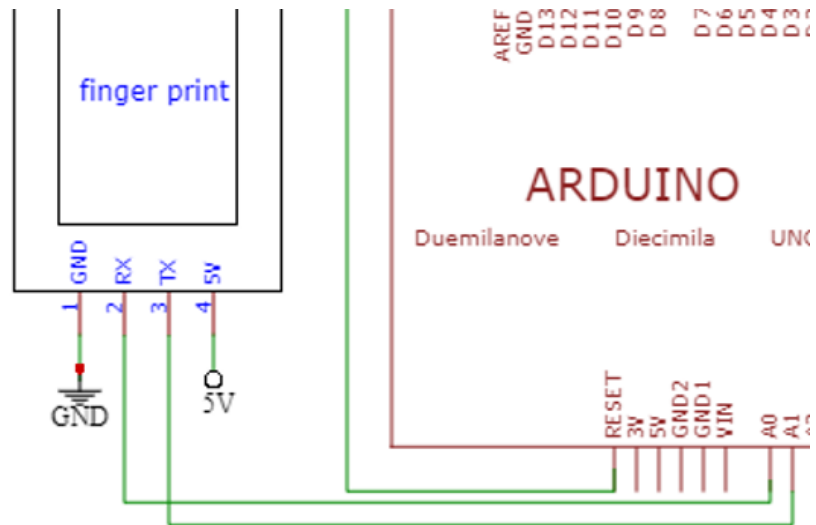
גם מי שאינם מנוסה בזיהוי טביעות אצבע מסוגל להבחין בהבדל בין שלושת הסוגים העיקריים של הטביעות: מערבולות, לולאות, וקשתות.

הסבר על הרכיב:
 הרכיב DY-05 חיישן טביעת אצבע הוא רכיב ביומטרי המאפשר זיהוי ואימות של טביעות אצבע משלב חיישן קפסטיבי וסורק תמונה, ומאפשר אחסון וזיהוי טביעות אצבע במספר קטן של צעדים.
 Arduino ומספק ממשק קל לשימוש עם מיקרובקרים כמו (UART) הוא תומך בתקשורת סריאלית הפרוטוקול מגדיר את פורמט העברת נתונים בין מחשב או מיקרו בקר למודול טביעת האצבע.
 UART. הפורמט מתאים לתקשורת טורית למערכת סורק טביעות אצבע יש שני שלבים של פעולה בסיסיים:
 - capture - קבלת תמונה של האצבע הנקראת גם צילום או לכידת - enrollment - א. הרשמה
 IMAGE. התמונה ויצירת קובץ תמונה הנקרא לקבוע אם תבנית הרכסים והעמקים בתמונה זו תואמת לתבנית של - matching - ב. התאמה
 רכסים ועמקים בתמונות שנסרקו מראש. ישנן 2 אפשרויות התאמה :
 טביעות N אומרת למצוא את טביעת האצבע שנמצאת כרגע בחיישן מתוך 1 N:1. התאמה האצבע במערכת.
 2. התאמה 1:1 אומרת האם טביעת האצבע שנמצאת כרגע בחיישן זהה לטביעת אצבע מסוימת. כלומר האם טביעת האצבע שבחיישן מתאימה לטביעת אצבע מספר 50 ששמורה במערכת.
 המשתמש צריך לשים אצבע בחלון החיישן פעמיים. המערכת מעבדת , enrollment בשלב ההרשמה
 את 2 התמונות שהתקבלו ותיצור תבנית של האצבע המבוססת על התוצאות ושומרת אותה בספרייה של המערכת.
 בשלב ההתאמה המשתמש שם את האצבע בחלון חיישן טביעת האצבע האופטי והמערכת יוצרת תבנית של האצבע ותשווה אותה לתבניות הנמצאות בספרייה של המערכת. בהתאמה של 1:1
 תתבצע התאמה עם כל N1 תתבצע התאמה עם תבנית מסוימת שנרצה מתוך הספרייה . בהתאמה התבניות שבספרייה . התשובה החוזרת ל 2 ההתאמות היא הצלחה או כשלון.

שימוש בחיישן טביעת אצבע בפרויקט

ישמש לאימות זהות המשתמש. משתמש יורשה לפתוח את הכספת במידה וטביעת האצבע נכונה.

8.1 חיבור הקורא למעבד



קורא האצבע מחובר למעבד בתקשורת טורית RS232 בקצב bau9600 לימוד קריאת האצבע נעשה באמצעות תוכנה במחשב המסופקת על ידי היצרן. את הקריאה נעשה על ידי הבקר.

8.2 תוכנית לקבלת טביעת האצבע

```
uint8_t getFingerprintEnroll()
{
    int p = -1;
    Serial.print("Waiting for valid finger to enroll as #");

    while (p != FINGERPRINT_OK) {
        p = finger.getImage
        switch (p) {
            case FINGERPRINT_OK:
                Serial.println("Image taken");
                LcdTouch.print(5, 20, "Remove finger and then", 2.6,
WHITE);
                LcdTouch.print(5, 40, "Place same finger again", 2.6,
WHITE);
                break;
            case FINGERPRINT_NOFINGER:
                Serial.println(".");
```

```

        break;
    case FINGERPRINT_PACKETRECIEVEERR:
        Serial.println("Communication error");
        LcdTouch.print(50, 160, "Communication error", 3,
GREEN);
        break;
    case FINGERPRINT_IMAGEFAIL:
        Serial.println("Imaging error");
        LcdTouch.print(50, 160, "Imaging error", 3, GREEN);
        break;
    default:
        Serial.println("Unknown error");
        LcdTouch.print(50, 160, "Unknown error", 3, GREEN);
        break;
    }
}

p = finger.image2Tz(1);
switch (p) {
    case FINGERPRINT_OK:
        Serial.println("Image converted");
        break;
    case FINGERPRINT_IMAGEMESS:
        Serial.println("Image too messy");
        LcdTouch.print(25, 160, "Image too messy", 3, GREEN);
        return p;
    case FINGERPRINT_PACKETRECIEVEERR:
        Serial.println("Communication error");
        return p;
    case FINGERPRINT_FEATUREFAIL:
        Serial.println("Could not find fingerprint features");
        LcdTouch.print(1, 160, "Could not find fingerprint
features", 3, GREEN);
        return p;
    case FINGERPRINT_INVALIDIMAGE:
        Serial.println("Could not find fingerprint features");
        return p;
    default:
        Serial.println("Unknown error");
        return p;
}

Serial.println("Remove finger");
delay(2000);

// Wait until the finger is removed
p = 0;
while (p != FINGERPRINT_NOFINGER) {

```

```

    p = finger.getImage();
}

LcdTouch.print(15, 100, "Found ID", 2.5, BLUE);
LcdTouch.print(15, 140, String(finger.fingerID), 2.5, BLUE);

Serial.println("Place same finger again");

while (p != FINGERPRINT_OK) {
    p = finger.getImage();
    switch (p) {
        case FINGERPRINT_OK:
            Serial.println("Image taken");
            break;
        case FINGERPRINT_NOFINGER:
            Serial.print(".");
            break;
        case FINGERPRINT_PACKETRECEIVEERR:
            Serial.println("Communication error");
            break;
        case FINGERPRINT_IMAGEFAIL:
            Serial.println("Imaging error");
            break;
        default:
            Serial.println("Unknown error");
            break;
    }
}

p = finger.image2Tz(2);
switch (p) {
    case FINGERPRINT_OK:
        Serial.println("Image converted");
        break;
    case FINGERPRINT_IMAGEMESS:
        Serial.println("Image too messy");
        return p;
    case FINGERPRINT_PACKETRECEIVEERR:
        Serial.println("Communication error");
        return p;
    case FINGERPRINT_FEATUREFAIL:
        Serial.println("Could not find fingerprint features");
        return p;
    case FINGERPRINT_INVALIDIMAGE:
        Serial.println("Could not find fingerprint features");
        return p;
    default:
        Serial.println("Unknown error");
}

```

```

        return p;
    }
    p = finger.createModel();
    if (p == FINGERPRINT_OK) {
        Serial.println("Prints matched!");
    } else if (p == FINGERPRINT_PACKETRECEIVEERR) {
        Serial.println("Communication error");
        return p;
    } else if (p == FINGERPRINT_ENROLLMISMATCH) {
        Serial.println("Fingerprints did not match");
        return p;
    } else {
        Serial.println("Unknown error");
        return p;
    }
    if (p == FINGERPRINT_OK) {
        Serial.println("Stored!");
    } else if (p == FINGERPRINT_PACKETRECEIVEERR) {
        Serial.println("Communication error");
        return p;
    } else if (p == FINGERPRINT_BADLOCATION) {
        Serial.println("Could not store in that location");
        return p;
    } else if (p == FINGERPRINT_FLASHERR) {
        Serial.println("Error writing to flash");
        return p;
    } else {
        Serial.println("Unknown error");
        return p;
    }
    while (!LcdTouch.touched());
}

```

9 תיאור פרויקט

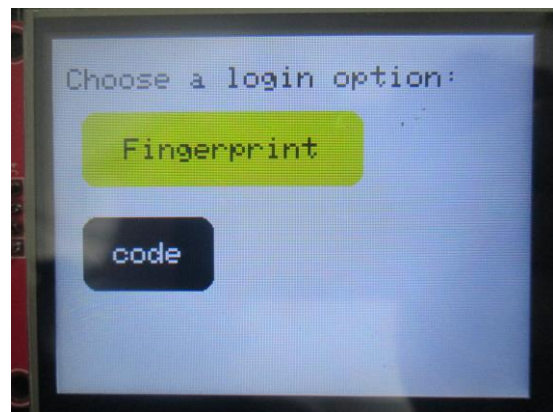
9.1 הסבר על הפרויקט

פרויקט זה עוסק בכספת חכמה.

בכניסה למערכת ישנם שני אפשרויות:

- כניסה למערכת באמצעות קוד.
 - כניסה למערכת באמצעות טביעת אצבע.
- באם הקוד או טביעת האצבע אינה נכונה – הכספת אינה תפתח ולד אדום יידלק.
- אם הקוד או טביעת האצבע נכונים – הכספת תפתח והלד יידלק בירוק.

9.2 צילומי מסך + הסבר



10 תיעוד הפרויקט

10.1 תהליך עבודה

לאחר שבחרתי בפרויקט הנוכחי למדתי על רכיביו ובחרתי באלו שהיו נראים לי המתאימים ביותר לפרויקט שלי, לאחר מכן התחלתי לעצב את המערכת מסכים התחלתיים וכדו', לאחר כל זאת התחלתי להתעסק עם הרכיבים, בתחילה חיברתי את בר הלדים ל Arduino בדקתי על תוכנית שזה עובד אח"כ המשכתי להלחמות הרכיבים טביעת האצבע, המנוע והלד.

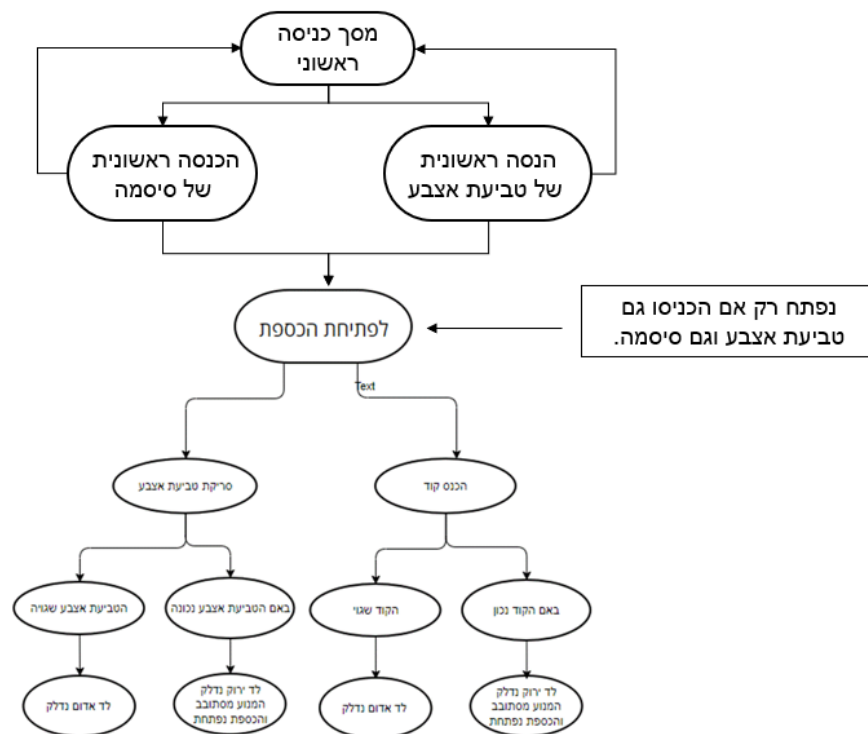
לכל רכיב כתבתי את הפונקציות המתאימות לו וכך הורכב הפרויקט.

10.2 תקלות ופתרונם

- התקלה: המנוע לא עבד טוב.
הפתרון: בדיקת כל החיבורים, גיליתי שחלקם לא היו מחוברים טוב אז חיברתי מחדש.
- התקלה: הלד אינו עבד טוב, הצבע האדום אינו פעל בנוסף בטעות חיברתי את האדמה לV5 (+) והלד התפוצץ.
הפתרון: החלפתי לד וזה עבד.
- התקלה: שום קוד לא עבד על הטביעת אצבע.
הפתרון: החיבור לא היה נכון וחלק מההלחמות התחברו.

11 תוכנה

11.1 תרשים זרימה



1.2 תוכניות

קוד זה הינו חלקי.

```
2. #include <Servo.h>
3. #include <SPI.h>
4. #include "TFT9341Touch.h"
5. tft9341touch LcdTouch(10, 9, 7, 2); //cs, dc ,tcs, tirq
6. #include <Adafruit_Fingerprint.h>
7. #include <SoftwareSerial.h>
8. enum statuses { success, failed_id, failed_sum, failed_size };
9.
10. int getFingerprintIDez();
11. // pin #2 is IN from sensor
12. // pin #3 is OUT from arduino
13. SoftwareSerial mySerial(A1,A0);
14. Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
15.
16. Servo myservo; // create servo object to control a servo
17. // twelve servo objects can be created on most boards
18. int pos = 0; // variable to store the servo position
19. uint8_t id; // מזהה טביעת אצבע
20. uint8_t result; // תוצאה של פעולות טביעת אצבע
21. int ButtonNum;
22. uint8_t getFingerprintEnroll();
23. int cursor2=0;
24. int string1[8];
25. int pass[8] = {1,2,3,4};
26. int indexString=0;
27. void setup() {
28.   LcdTouch.begin();
29.   LcdTouch.clearButton();
30.   LcdTouch.setRotation(0); //0,1,2,3
31.   LcdTouch.setTextSize (2);
32.   LcdTouch.setCursor (40, 40);
33.   LcdTouch.set(3780, 372, 489, 3811); //Calibration
34.
35.   screenMain();
36. }
37.
38. void loop()
39. {
40.   uint16_t x, y;
41.   String str;
42.   ButtonNum = 0;
43.
44.   if (LcdTouch.touched())
45.   {
```

```

46.    LcdTouch.readTouch();
47.    x = LcdTouch.xTouch;
48.    y = LcdTouch.yTouch;
49.    ButtonNum = LcdTouch.ButtonTouch(x, y);
50.    // str="x=" + String(x) + " y=" + String(y) + " Bu=" +
    String(ButtonNum);
51.    // LcdTouch.print (25, 220, str, 2, BLACK, CYAN);
52.    // LcdTouch.fillCircle(x, y, 3, WHITE);
53.    if (ButtonNum == 1)
54.    {
55.        choosKay();
56.        // screenMain();
57.    }
58.    else if (ButtonNum == 2)
59.    {
60.        Fingerprint();
61.        // screen1();
62.    }
63.    else if (ButtonNum == 3 )
64.    {
65.        numberBoard();
66.        // screen1();
67.    }
68.    else if (ButtonNum>4)
69.    {
70.        chackEnd(ButtonNum);
71.    }
72.    }
73. }
74.}
75.
76.// x:\הנדסאים שנה ב תשפד\מערכות משובצות\חומרים לפרויקט\תכניות לרכיבים\טביעת
    אצבע\Adafruit-Fingerprint-Sensor-Library-
    master\Adafruit_Fingerprint.cpp
77.void screenMain()
78.{
79.    LcdTouch.fillScreen (BLACK);
80.    LcdTouch.print (30 ,20, "Welcome to the", 2, WHITE);
81.    LcdTouch.print (10,45,"smart safe system", 2, WHITE);
82.    LcdTouch.drawButton(1 , 120, 100, 90, 50, 10, BLUE, WHITE,
        "start", 2); // NumButton, x, y, width, height, r, Color,
        textcolor, label, textsize);
83.    // digitalWrite(5, 255);
84.    // digitalWrite(8,255);
85.}
86.void choosKay()
87.{
88.    LcdTouch.fillScreen (BLACK);

```

```

89.  LcdTouch.print(10, 20, "Choose a login option:", 2, WHITE);
90.
91.  LcdTouch.drawButton(2, 20, 50, 190, 50, 10, BLUE, WHITE,
    "Finger print", 2); // NumButton, x, y, width, height, r, Color,
    textcolor, label, textsize);
92.
93.  LcdTouch.drawButton(3, 20, 120, 90, 50, 10, BLUE, WHITE,
    "code", 2);
94.    while (!LcdTouch.touched());
95.
96.}
97.

98.void Fingerprint() {
99.  LcdTouch.fillScreen (WHITE);
100.    LcdTouch.setTextColor(BLACK);
101.    LcdTouch.setTextSize (3);
102.    LcdTouch.setCursor (25, 30);
103.    LcdTouch.print (60, 20, "Place your finger", 2, BLACK);
104.    LcdTouch.print (80, 45, "on the sensor", 2, BLACK);
105.    LcdTouch.setTextSize (1);
106.    LcdTouch.begin();
107.    Serial.begin(9600);
108.    finger.begin(57600);
109.    LcdTouch.clearButton();
110.    LcdTouch.setRotation(0); //0,1,2,3
111.    LcdTouch.setTextSize(2);
112.    LcdTouch.setCursor(40, 40);
113.    LcdTouch.set(3780, 372, 489, 3811); //Calibration
114.    while (!Serial);
115.    delay(500);
116.    Serial.begin(9600);
117.    Serial.println("Adafruit Fingerprint sensor enrollment");
118.    // set the data rate for the sensor serial port
119.    finger.begin(57600);
120.    if (finger.verifyPassword())
121.    {
122.        Serial.println("Found fingerprint sensor!");
123.    }
124.    else
125.    {
126.        Serial.println("Did not find fingerprint sensor :(");
127.        while (1) ;
128.    }
129.    // load();
130.    //check();
131.    getFingerprintEnroll();
132.    screenMain();
133.    }

```

```

134.
135.     void numberBoard()
136.     {
137.
138.         LcdTouch.fillScreen (BLACK);
139.         LcdTouch.clearButton();
140.         LcdTouch.drawButton(5, 30, 30, 40, 40, 2, RED, WHITE,
    "1", 4); // NumButton, x, y, width, height, r, Color, textcolor,
    label, textsize);
141.         LcdTouch.drawButton(6, 75, 30, 40, 40, 2, RED, WHITE,
    "2", 4);
142.         LcdTouch.drawButton(7, 120, 30, 40, 40, 2, RED, WHITE,
    "3", 4);
143.         LcdTouch.drawButton(8, 30, 75, 40, 40, 2, RED, WHITE,
    "4", 4);
144.         LcdTouch.drawButton(9, 75, 75, 40, 40, 2, RED, WHITE,
    "5", 4);
145.         LcdTouch.drawButton(10, 120, 75, 40, 40, 2, RED, WHITE,
    "6", 4);
146.         LcdTouch.drawButton(11, 30, 120, 40, 40, 2, RED, WHITE,
    "7", 4);
147.         LcdTouch.drawButton(12, 75, 120, 40, 40, 2, RED, WHITE,
    "8", 4);
148.         LcdTouch.drawButton(13, 120, 120, 40, 40, 2, RED, WHITE,
    "9", 4);
149.         LcdTouch.drawButton(14, 75, 165, 40, 40, 2, RED, WHITE,
    "0", 4);
150.         LcdTouch.drawButton(15, 30, 165, 40, 40, 2, RED, WHITE,
    "<-", 2);
151.         LcdTouch.drawButton(16, 120, 165, 40, 40, 2, RED, WHITE,
    "ok", 2);
152.         while(!LcdTouch.touched());
153.
154.     }
155.
156.     void keyboard(int ButtonNum)
157.     {
158.         switch(ButtonNum)
159.         {
160.             case 5: LcdTouch.setCursor (cursor2+=15,210);
161.                     LcdTouch.println ("1");
162.                     string1[indexString++]=1;
163.                     delay(500);
164.                     break;
165.             case 6: LcdTouch.setCursor (cursor2+=15,210);
166.                     LcdTouch.println ("2");
167.                     string1[indexString++]=2;
168.                     delay(500);

```

```

169.         break;
170.     case 7: LcdTouch.setCursor (cursor2+=15,210);
171.             LcdTouch.println ("3");
172.             string1[indexString++]=3;
173.             delay(500);
174.             break;
175.     case 8: LcdTouch.setCursor (cursor2+=15,210);
176.             LcdTouch.println ("4");
177.             string1[indexString++]=4;
178.             delay(500);
179.             break;
180.     case 9: LcdTouch.setCursor (cursor2+=15,210);
181.             LcdTouch.println ("5");
182.             string1[indexString++]=5;
183.             delay(500);
184.             break;
185.     case 10: LcdTouch.setCursor (cursor2+=15,210);
186.             LcdTouch.println ("6");
187.             string1[indexString++]=6;
188.             delay(500);
189.             break;
190.     case 11: LcdTouch.setCursor (cursor2+=15,210);
191.             LcdTouch.println ("7");
192.             string1[indexString++]=7;
193.             delay(500);
194.             break;
195.     case 12: LcdTouch.setCursor (cursor2+=15,210);
196.             LcdTouch.println ("8");
197.             string1[indexString++]=8;
198.             delay(500);
199.             // LcdTouch.setCursor (cursor2+=15,210);
200.             break;
201.     case 13: LcdTouch.setCursor (cursor2+=15,210);
202.             LcdTouch.println ("9");
203.             string1[indexString++]=9;
204.             delay(500);
205.             break;
206.     case 14: LcdTouch.setCursor (cursor2+=15,210);
207.             LcdTouch.println ("0");
208.             string1[indexString++]=0;
209.             delay(500);
210.             break;
211.     case 15: LcdTouch.setCursor (cursor2,210);
212.             LcdTouch.println ("");
213.             cursor2-=15;
214.             LcdTouch.setCursor (cursor2,210);
215.             indexString--;
216.             delay(500);

```

```

217.             break;
218.         // case 16: screen1();
219.         // break;
220.     }
221.     while(!LcdTouch.touched());
222. }
223.
224. void chackEnd(int ButtonNum)
225. {
226.     int i=0,cursor1=10,x1,y1;
227.     while(ButtonNum!=16)
228.     {
229.         keyboard(ButtonNum);
230.         LcdTouch.readTouch();
231.         x1 = LcdTouch.xTouch;
232.         y1 = LcdTouch.yTouch;
233.         ButtonNum = LcdTouch.ButtonTouch(x1, y1);
234.     }
235.     if (ButtonNum==16){
236.         delay(1000);
237.         LcdTouch.setTextColor(WHITE,BLACK);
238.         LcdTouch.setCursor (cursor1,210);
239.         while(string1[i]!=NULL)
240.         {
241.             LcdTouch.print(String(string1[i++]));
242.             cursor1+=15;
243.             LcdTouch.setCursor(cursor1,210);
244.         }
245.         IsSecsses(string1);
246.         // for (int i =0;i<=indexString;i++)
247.         // {
248.         //     string1[i]='\0';
249.         //     keyboard(15);
250.
251.         // }
252.         // if(strcmp(pass,string1)==0)
253.         // {
254.         // }
255.         choosKay();
256.     }
257. }
258.
259. int IsSecsses(int string1[])
260. {
261.     int g=0;
262.     LcdTouch.fillScreen (BLACK);
263.     for (int i = 0;i<8;i++)
264.     {

```



```

265.         if(pass[i]!=string1[i])
266.         {
267.             g=1;
268.             break;
269.         }
270.     }
271.     if(g==0)
272.     {
273.         LcdTouch.println ("success");
274.     }
275.     else{
276.         LcdTouch.println ("dont secess");
277.         delay(500);
278.         numberBoard();
279.     }
280. }
281. for (int i =0;i<8;i++)
282.     LcdTouch.print(string1[i]);
283. for (int i =0;i<indexString;i++)
284.     keyboard(15);
285. indexString=0;
286.
287.     while(!LcdTouch.touched());
288.
289. }
290. uint8_t getFingerprintEnroll()
291. {
292.     int p = -1;
293.     Serial.print("Waiting for valid finger to enroll as
294. #");
295.     // Loop until a valid fingerprint image is obtained
296.     while (p != FINGERPRINT_OK) {
297.         p = finger.getImage(); // Attempt to capture
298.         fingerprint image
299.
300.         switch (p) {
301.             case FINGERPRINT_OK:
302.                 Serial.println("Image taken");
303.                 // Display instructions on an LCD (assuming
304.                 LcdTouch object is defined)
305.                 LcdTouch.print(5, 20, "Remove finger and
306. then", 2.6, WHITE);
307.                 LcdTouch.print(5, 40, "Place same finger
308. again", 2.6, WHITE);
309.                 break;
310.             case FINGERPRINT_NOFINGER:
311.                 Serial.println(".");

```

```

308.                // Optionally display a dot or message on
LCD
309.                break;
310.            case FINGERPRINT_PACKETRECIEVEERR:
311.                Serial.println("Communication error");
312.                LcdTouch.print(50, 160, "Communication
error", 3, GREEN);
313.                break;
314.            case FINGERPRINT_IMAGEFAIL:
315.                Serial.println("Imaging error");
316.                LcdTouch.print(50, 160, "Imaging error", 3,
GREEN);
317.                break;
318.            default:
319.                Serial.println("Unknown error");
320.                LcdTouch.print(50, 160, "Unknown error", 3,
GREEN);
321.                break;
322.        }
323.    }
324.
325.    // Once a fingerprint image is obtained, convert it
326.    p = finger.image2Tz(1);
327.    switch (p) {
328.        case FINGERPRINT_OK:
329.            Serial.println("Image converted");
330.            break;
331.        case FINGERPRINT_IMAGEMESS:
332.            Serial.println("Image too messy");
333.            LcdTouch.print(25, 160, "Image too messy", 3,
GREEN);
334.            return p;
335.        case FINGERPRINT_PACKETRECIEVEERR:
336.            Serial.println("Communication error");
337.            return p;
338.        case FINGERPRINT_FEATUREFAIL:
339.            Serial.println("Could not find fingerprint
features");
340.            LcdTouch.print(1, 160, "Could not find
fingerprint features", 3, GREEN);
341.            return p;
342.        case FINGERPRINT_INVALIDIMAGE:
343.            Serial.println("Could not find fingerprint
features");
344.            return p;
345.        default:
346.            Serial.println("Unknown error");
347.            return p;

```

```

348.     }
349.
350.     Serial.println("Remove finger");
351.     delay(2000);
352.
353.     // Wait until the finger is removed
354.     p = 0;
355.     while (p != FINGERPRINT_NOFINGER) {
356.         p = finger.getImage();
357.     }
358.
359.     // Serial.print("ID "); Serial.println(id);
360.     LcdTouch.print(15, 100, "Found ID", 2.5, BLUE);
361.     // LcdTouch.print(15, 120, String(id), 2.5, BLUE);
362.     LcdTouch.print(15, 140, String(finger.fingerID), 2.5,
BLUE);
363.
364.     Serial.println("Place same finger again");
365.
366.     // Loop until a valid fingerprint image is obtained
again
367.     while (p != FINGERPRINT_OK) {
368.         p = finger.getImage();
369.         switch (p) {
370.             case FINGERPRINT_OK:
371.                 Serial.println("Image taken");
372.                 break;
373.             case FINGERPRINT_NOFINGER:
374.                 Serial.print(".");
375.                 break;
376.             case FINGERPRINT_PACKETRECEIVEERR:
377.                 Serial.println("Communication error");
378.                 break;
379.             case FINGERPRINT_IMAGEFAIL:
380.                 Serial.println("Imaging error");
381.                 break;
382.             default:
383.                 Serial.println("Unknown error");
384.                 break;
385.         }
386.     }
387.
388.     // Convert the second fingerprint image
389.     p = finger.image2Tz(2);
390.     switch (p) {
391.         case FINGERPRINT_OK:
392.             Serial.println("Image converted");
393.             break;

```

```

394.         case FINGERPRINT_IMAGEMESS:
395.             Serial.println("Image too messy");
396.             return p;
397.         case FINGERPRINT_PACKETRECIEVEERR:
398.             Serial.println("Communication error");
399.             return p;
400.         case FINGERPRINT_FEATUREFAIL:
401.             Serial.println("Could not find fingerprint
features");
402.             return p;
403.         case FINGERPRINT_INVALIDIMAGE:
404.             Serial.println("Could not find fingerprint
features");
405.             return p;
406.         default:
407.             Serial.println("Unknown error");
408.             return p;
409.     }
410.
411.     // Create a fingerprint model
412.     // Serial.print("Creating model for #");
Serial.println(id);
413.     p = finger.createModel();
414.     if (p == FINGERPRINT_OK) {
415.         Serial.println("Prints matched!");
416.     } else if (p == FINGERPRINT_PACKETRECIEVEERR) {
417.         Serial.println("Communication error");
418.         return p;
419.     } else if (p == FINGERPRINT_ENROLLMISMATCH) {
420.         Serial.println("Fingerprints did not match");
421.         return p;
422.     } else {
423.         Serial.println("Unknown error");
424.         return p;
425.     }
426.
427.     // Store the fingerprint model
428.     // Serial.print("ID "); Serial.println(id);
429.     // p = finger.storeModel(id);
430.     if (p == FINGERPRINT_OK) {
431.         Serial.println("Stored!");
432.     } else if (p == FINGERPRINT_PACKETRECIEVEERR) {
433.         Serial.println("Communication error");
434.         return p;
435.     } else if (p == FINGERPRINT_BADLOCATION) {
436.         Serial.println("Could not store in that location");
437.         return p;
438.     } else if (p == FINGERPRINT_FLASHERR) {

```

```
439.         Serial.println("Error writing to flash");
440.         return p;
441.     } else {
442.         Serial.println("Unknown error");
443.         return p;
444.     }
445.
446.     // Wait until touch input is detected to continue
447.     while (!LcdTouch.touched());
448.
449.     // End of function
450. }
```

11.3 הסבר פונקציות

שם הפונקציה	תפקידה	ערכים שהיא מקבלת או מחזירה
screenMain()	מסך ראשי עם כפתור התחלה.	אינה מקבלת ומחזירה.
void choosKay()	מסך שבו יש 2 כפתורים. 1. פתיחת הכספת ע"י טביעת אצבע. 2. פתיחת הבספת ע"י קוד.	אינה מקבלת או מחזירה.
void numberBoard ()	מציירת על המסך לוח מקשי מספרים.	אינה מקבלת או מחזירה.
Void keyboard(int ButtonNum)	מקבלת את הכפתור שבו לחצו משרשרת אותו למחרוזת הקוד, ומציירת אותו על המסך.	מקבלת כפתור, אינה מחזירה.
Void chackEnd (int ButtonNum)	מקבל כפתור ובודק כל עוד לא נלחץ על סיום הסיסמה הוא ממשיך לשלוח לפונקציה של הלוח מקשים. בסיום יש לה את מחרוזת הסיסמה. הפונקציה שולחת לבדיקה את הבסיסמה לפונקציה IsSecsses()	מקבלת את הכפתור שלחץ.
int IsSecsses(int string1[])	הפונקציה מקבלת סיסמה ובודקת האם היא נכונה. אם כן היא שולחת לפונקציה שתפתח את הכספת.	מקבלת מחרוזת ולא מחזירה.
uint8_t getFingerprintEnroll()	פונקציה המכניסה בפעם הראשונה טביעת אצבע למערכת.	אינה מקבלת, מחזירה את הטביעת אצבע.
()int getCode	פונקציה המכניסה בפעם הראשונה קוד למערכת.	אינה מקבלת, מחזירה את הקוד.

אינה מקבלת או מחזירה.	פונקציה המפעילה את המנוע 90 מעלות – פותחת את הכספת.	int openSafe()
אינה מקבלת או מחזירה.	פונקציה המפעילה את חיישן טביעת האצבע, מקבלת טביעת אצבע ובודקת האם היא נכונה. אם כן היא שולחת לפונקציה שפותחת את הכספת.	uint8_t getFingerPrint()

12 סיכום

לפרויקט זה ניתן להוסיף מרכיבים רבים, כמו:

- משמיע צליל- כאשר הכספת נפתחת.
 - מנוע חזק ועצמתי למנעול .
 - רטט כאשר הקוד או טביעת האצבע שגויים הרטט יפעל.
- ועוד המון שכלולים.....

13 תקשורת SPI

תקשורת טורית סינכרונית, (Serial Peripheral Interface) בין שני התקנים Mater-Slave כאשר ה-Master מספק clk למערכת וה-Slave שולח/מקבל מידע לפי שיעון זה. מערכת ה-SPI היא מערכת דופלקס מלאה, (Full Duplex) כלומר, שליחת מידע וקבלת מידע (דרך 2 חוטים) בו זמנית.

על מנת להשתמש בתקשורת SPI יש צורך בארבעה קווים, 2 קווי בקרה ו-2 קווי מידע/

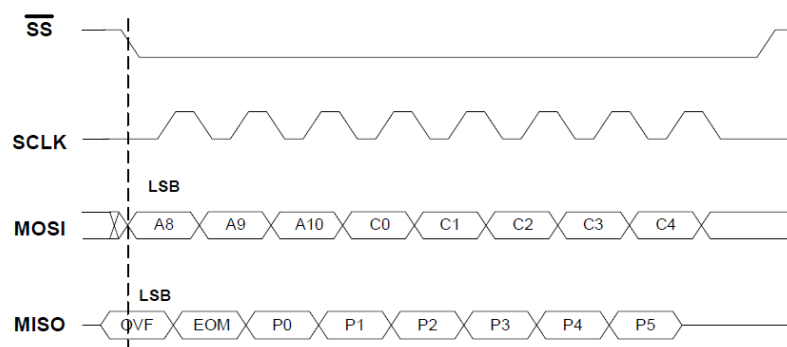
MOSI (Master Out Slave In) - מידע נשלח מה-Master אל ה-Slave.

MISO (Master In Slave Out) - מידע נשלח מה-Slave אל ה-Master.

SCLK - שיעון טורי- קובע את תזמון העברת המידע בין Master וה-Slave

SS (Slave Select) - קו זה פעיל בנמוך ומספק בקרה להתחלת פעולת התקשורת עם ה-Slave.

שליחת מילת בקרה של 8 סיביות



בתחילת התקשורת, הדק SS יורדת ל-0. לאחר מכן מועבר המידע מה-Master דרך הדק MOSI. במקביל ה-Master יכול לקרוא את המידע מה-Slave דרך הדק MISO. בסיום התקשורת הדק SS חוזרת ל-1 לוגי.